Tear everything down

Internal Developer Platform runbook. Stops the AWS bill and keeps your code. About 25 minutes, mostly waiting.

Run every command in **Git Bash**, starting from the project folder. Do the steps in order.

```
cd ~/OneDrive/Desktop/IDP\ Project
aws eks update-kubeconfig --name idp-cluster --region eu-north-1
```

> **This deletes the portal's database**, which holds the list of environments and login sessions. Every running environment is deleted too. Your code, the Helm chart and the Terraform files are safe in Git.

Why not just run `terraform destroy`? Kubernetes creates the load balancer and the database disk, not Terraform, so Terraform doesn't know about them. If they still exist, the destroy gets stuck trying to delete the network underneath them. Steps 1 to 3 remove them first.

---

1. Stop ArgoCD from recreating things

```
kubectl delete applications --all -n argocd --wait=false
```

---

2. Delete the load balancer and the database disk

```
kubectl delete ingress --all -A --timeout=180s
kubectl delete statefulset postgres -n idp-project --ignore-not-found
kubectl delete pvc --all -n idp-project
```

---

3. Wait until AWS confirms they're gone 1 to 3 minutes

Paste the whole block. It repeats until both are 0.

```
while true; do
  LB=$(aws elbv2 describe-load-balancers --region eu-north-1 \
    --query 'length(LoadBalancers)' --output text)
  VOL=$(aws ec2 describe-volumes --region eu-north-1 \
    --filters Name=status,Values=available \
    --query 'length(Volumes)' --output text)
  echo "load balancers=$LB detached disks=$VOL"
  [ "$LB" = "0" ] && [ "$VOL" = "0" ] && break
  sleep 15
done
```

> Expected: `load balancers=0 detached disks=0`

4. Destroy the infrastructure about 20 minutes

```
cd terraform
terraform destroy -auto-approve
cd ..
```

> Expected at the end: `Destroy complete! Resources: 66 destroyed.`

> **Keep the PC awake and online until it finishes.** If it's interrupted or fails with a network error, nothing is lost, but you keep paying until it's finished: run `terraform destroy -auto-approve` again and it continues from where it stopped.

5. Confirm nothing billable is left

Paste the whole block.

```
R=eu-north-1
echo "clusters:  $(aws eks list-clusters --region $R \
  --query clusters --output text)"
echo "instances: $(aws ec2 describe-instances --region $R \
  --filters Name=instance-state-name,Values=pending,running,stopping,stopped \
  --query 'Reservations[].Instances[].InstanceId' --output text)"
echo "ALBs:      $(aws elbv2 describe-load-balancers --region $R \
  --query 'LoadBalancers[].LoadBalancerName' --output text)"
echo "disks:     $(aws ec2 describe-volumes --region $R \
  --query 'Volumes[].VolumeId' --output text)"
echo "NAT:       $(aws ec2 describe-nat-gateways --region $R \
  --filter Name=state,Values=pending,available \
  --query 'NatGateways[].NatGatewayId' --output text)"
echo "IPs:       $(aws ec2 describe-addresses --region $R \
  --query 'Addresses[].PublicIp' --output text)"
echo "Redis:     $(aws elasticache describe-cache-clusters --region $R \
  --query 'CacheClusters[].CacheClusterId' --output text)"
echo "ECR:       $(aws ecr describe-repositories --region $R \
  --query 'repositories[].repositoryName' --output text)"
```

> Expected: every line empty after its label.

What stays, on purpose

Item	Cost	Why it stays
Route 53 hosted zone for idplatform.tech	$0.50 a month	The domain's name servers point at it. Deleting it means reconfiguring the domain and validating the certificate again.
ACM certificate for idplatform.tech and *.idplatform.tech	Free	Reused on the next apply.
GitHub OAuth Apps	Free	Reused on the next apply.
Everything in Git, and your secret files	Free	Your project.

The site is unreachable until the next apply. A leftover DNS record for `idplatform.tech` is harmless: ExternalDNS updates it after the next apply. Folders in `environments/` for environments that weren't destroyed through the portal stay in Git and are also harmless.

If something goes wrong

What you see	What to do
`no such host` from kubectl	Run the `update-kubeconfig` command at the top of this runbook. If the cluster no longer exists, skip to step 4.
`no such host` or a timeout from Terraform	A network blip. Run the destroy again.
`Error acquiring the state lock`	An earlier run was interrupted. Run `terraform force-unlock -force <ID>` with the ID it prints, then destroy again.
Destroy hangs on the VPC, subnets or internet gateway, or `DependencyViolation`	A load balancer or its security group is still there. Repeat steps 1 to 3, then destroy again. Look for leftover `k8s-*` entries under EC2 → Load Balancers and Security Groups in the AWS console.
`kubectl delete ingress` hangs	The controller may already be gone. Press Ctrl+C and look under EC2 → Load Balancers. If one is there, delete it in the console and then continue with step 3.

Coming back

Follow the **Bring everything up** runbook (`docs/runbook-apply.pdf`).